

VIT GUIDE

A Python-Based Campus Information and Utility System

PROJECT REPORT

Submitted by

Abhishek Kumar

Reg. No.: 26BCE10471

VIT Bhopal University

Academic Year: 2026–2027

Department of Computer Science / School of Computing

1. Abstract

VIT Guide is a menu-driven Python application designed to provide students with quick access to useful campus information through a simple terminal interface. The system organizes information about the Special Block, boys hostel, sports courts, shops, AB1 building, searchable campus items, and user favourites. It also includes food ordering and item purchasing simulations with automatic bill calculation.

The project demonstrates fundamental Python programming concepts such as dictionaries, lists, functions, loops, conditional statements, string handling, user input validation, list operations, and modular program design. The application is intentionally lightweight and does not require a database, web server, external API, or graphical interface.

2. Introduction

Students frequently need basic campus information such as where a shop is located, what food is available, which sports courts exist, or where a hostel block is situated. VIT Guide addresses this need by bringing common campus information into one interactive terminal application.

The project focuses on simplicity and usability. A student selects options from numbered menus and receives the required information. The same program also provides practical examples of how Python can be used to build a small information-management system.

3. Problem Statement

Campus information may be spread across different sources, making it inconvenient for a student to quickly find simple information. A lightweight application is required that can organize frequently used campus information and allow students to search, save favourites, view prices, and calculate bills without depending on complex software infrastructure.

4. Objectives

- Create a simple terminal-based VIT campus guide.
- Organize campus places and information using Python data structures.
- Provide menus for food, shops, sports courts, hostel blocks, and AB1.
- Implement a search feature for places, blocks, courts, and shop items.
- Allow users to add and remove favourites.
- Provide an ordering system with quantities and automatic total calculation.
- Practice modular programming through reusable Python functions.
- Build a project that is easy to understand, run, modify, and extend.

5. Scope of the Project

The current version is a local terminal application. It stores information directly in Python variables and dictionaries. It is intended as an educational prototype rather than an official VIT information system. Future versions could connect the same interface to a database or web/mobile frontend after appropriate

verification of campus information.

6. Technologies Used

Technology / Concept	Purpose
Python 3	Core programming language
Dictionaries	Store menus, prices, locations, and structured information
Lists	Store courts, cafes, favourites, and shop locations
Functions	Divide the application into reusable modules
Loops	Repeat menus and validate choices
Conditional statements	Control application flow
String methods	Search and compare user input
Terminal / Command Prompt	User interface

7. System Features

Special Block

Displays listed campus food/service locations and their menus. Users can view menus, place sample orders, and save locations to favourites.

Boys Hostel

Displays hostel blocks and basic location information.

Sports Courts

Lists basketball, volleyball, football, badminton, and tennis courts and allows a selected court to be added to favourites.

Shops

Shows shop locations, item prices, and a simple purchasing interface.

AB1 Building

Displays the proctor cabin and listed cafes.

Search

Searches across places, courts, shops, hostel blocks, and shop items using partial text matching.

My Favourites

Stores selected places/courts and allows a favourite to be removed.

Bill Calculation

Calculates the total cost using item price multiplied by quantity.

8. Data Structures

The project uses dictionaries for structured information. For example, the `special_block` dictionary maps a location to a menu dictionary containing item names and prices. Lists are used where an ordered collection is sufficient, such as the courts list and favourites list.

This design keeps the project easy to read for a beginner while demonstrating nested dictionaries, list indexing, dictionary keys, iteration, and mutable data.

9. Program Architecture

The program follows a simple modular structure. The `main()` function acts as the entry point and displays the main menu. Each major campus area has its own function. Smaller reusable functions such as `ask_number()`, `title()`, `pause()`, `show_list()`, `add_favourite()`, and `make_order()` reduce repetition.

- `main()` → controls the complete application flow.
- `special_block_section()` → displays Special Block locations.
- `place_options()` → provides actions for an individual place.
- `make_order()` → handles item selection, quantities, and billing.
- `hostel_section()` → displays boys hostel information.
- `courts_section()` → displays sports courts and favourites.
- `shops_section()` → handles shops, prices, and purchases.
- `ab1_section()` → displays AB1 information.
- `search_section()` → performs text-based search.
- `favourites_section()` → displays and removes favourites.

10. Important Functions and Logic

10.1 Input Validation

`ask_number()` repeatedly asks the user for input until a numeric value is entered. The `isdigit()` method prevents non-numeric menu choices from being passed directly to `int()`.

10.2 Ordering Logic

`make_order()` converts the selected menu dictionary keys into a list. The user selects an item number and enters a quantity. The program calculates each line cost using $\text{price} \times \text{quantity}$ and adds it to the running total.

10.3 Favourites

`add_favourite()` checks whether a name is already present in the favourites list. If it is not present, `append()` adds it. The favourites section uses `pop()` to remove an item selected by its number.

10.4 Search

`search_section()` converts the user's search term to lowercase and checks whether that term occurs inside the names of places, courts, shops, hostel blocks, and shop items. This enables partial and case-insensitive searching.

11. Sample User Flow

1. Program starts → user enters their name → main menu appears.
2. User selects Special Block → selects Bistro → views menu or places an order.
3. User selects an item and quantity → program calculates the bill.
4. User returns to the main menu → opens Sports Courts → selects Badminton.
5. User can save Badminton court to favourites.
6. User opens Search → enters "water" → matching shop item(s) are displayed.
7. User opens My Favourites → views or removes saved entries.
8. User selects Exit → program displays a closing message.

12. Algorithm

- Start the program.
- Ask the user for their name.
- Display the main menu.
- Read and validate the user's choice.
- Open the selected campus section.
- Display information or perform the selected operation.
- Return to the main menu after completing a section.
- Continue until the user selects Exit.
- Display the final message and stop.

13. Testing

Test Case	Input / Action	Expected Result
Main menu	Choose 1	Special Block opens
Invalid menu	Enter letters	Program asks for a number
Food order	Burger × 2	Adds Rs 160 to total
Favourites	Add Bistro	Bistro appears in favourites
Duplicate favourite	Add Bistro again	Duplicate warning appears
Search	Enter 'water'	Matching item/place is displayed
Remove favourite	Choose valid number	Selected favourite is removed
Exit	Choose 0	Program ends with goodbye message

14. Advantages

- Simple and beginner-friendly terminal interface.
- No database or internet connection is required.

- Modular functions make the code easier to understand and modify.
- Demonstrates multiple core Python concepts in one project.
- Includes practical features rather than only static information.
- Can be expanded into a GUI, web application, or database-backed system.

15. Limitations

- The information is stored statically in the source code.
- There is no persistent database, so favourites are lost when the program closes.
- The project does not provide real-time campus availability or booking.
- Prices and locations must be manually updated in the Python code.
- There is no authentication or separate account system.
- The current interface is terminal-based only.

16. Future Enhancements

- Add a SQLite/MySQL database for persistent data.
- Create student login and personal profiles.
- Save favourites permanently.
- Add real-time opening hours after verifying official data.
- Add a graphical or web interface.
- Add admin functionality to update menus and prices.
- Add campus navigation or map integration.
- Add order history and printable receipts.
- Add robust exception handling and unit tests.

17. Learning Outcomes

By completing this project, the student demonstrates practical understanding of Python fundamentals. The project applies variables, dictionaries, lists, functions, loops, if/elif/else conditions, string methods, indexing, `append()`, `pop()`, `input()`, `print()`, and basic validation in a single working application.

It also demonstrates decomposition of a larger problem into smaller functions. This is an important programming skill because individual modules can be tested, understood, and modified independently.

18. Conclusion

VIT Guide is a compact but functional Python project that combines campus information, searching, favourites, menus, ordering, and billing into one terminal application. Its main value is educational: it shows how basic Python concepts can be combined to create a complete interactive program.

The project provides a foundation for future development. With persistent storage, verified campus data, authentication, and a graphical interface, the same core logic could evolve into a more advanced campus utility application.

19. References

1. Python 3 documentation and standard library concepts.
2. Python programming concepts practiced through classroom/laboratory exercises.
3. VIT Bhopal campus information used as project data. Campus-specific details should be verified with official sources before deployment as a real campus service.

20. Project Details

Project Title	VIT Guide – Campus Information and Utility System
Student	Abhishek Kumar
Registration Number	26BCE10471
Institution	VIT Bhopal University
Language	Python
Interface	Terminal / Command Line
Project Type	Academic Mini Project
Academic Year	2026–2027